

E-Learning Student Course Enrollment Database Using SQL

Database Management System | Case Study 162

Submitted by: Dharmit Sathvara

Roll No: 150096725080

1. Problem Statement

An e-learning platform offers multiple courses to thousands of learners. Students can enroll in several courses, and each course has many students. The database must record who is enrolled in which course, when they enrolled and how far they have progressed. It must answer questions such as:

- Which courses is each student enrolled in?
- How many students are enrolled in each course?
- Which students are enrolled in more than two courses?
- Which courses have no enrollments at all?
- How is a student's course completion status updated?

Objectives

- Create the students, courses and enrollments tables.
- Use enrollments as a junction table that stores the enrollment date and completion status.
- Implement a proper many-to-many relationship using primary keys, foreign keys and constraints.
- Write queries using JOIN, LEFT JOIN, GROUP BY, HAVING, NOT EXISTS and UPDATE in PostgreSQL.
- Seed the database with realistic sample data and record every design decision with the alternative rejected.

Project files

File	Purpose
ER.png	Entity-relationship diagram
schema.sql	Tables, primary and foreign keys, constraints
db.sql	Sample data (INSERT statements)
query.sql	Queries with comments and output

2. Entity-Relationship Diagram

The model has three tables. One student can have many enrollments, and one course can have many enrollments. Each enrollment belongs to exactly one student and exactly one course. Together, these

two one-to-many relationships create a many-to-many relationship between students and courses, resolved by the junction table enrollments.

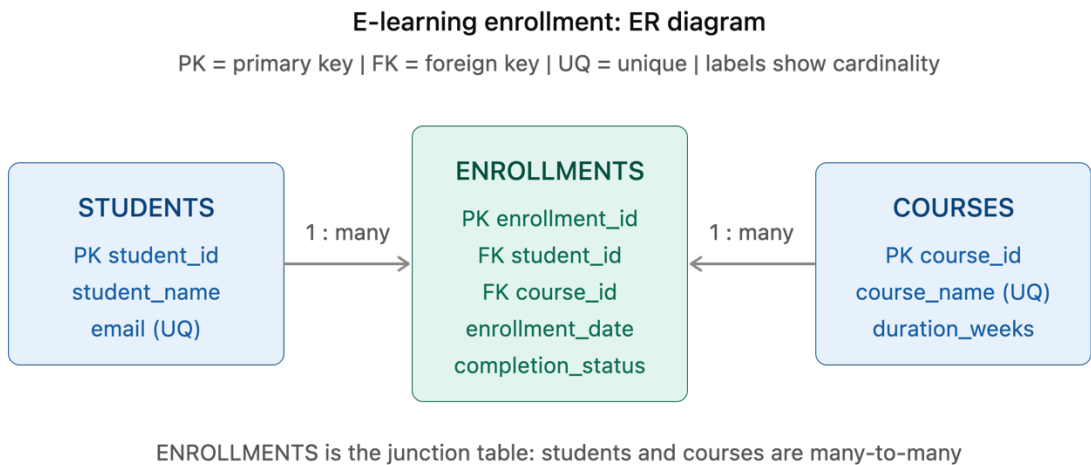
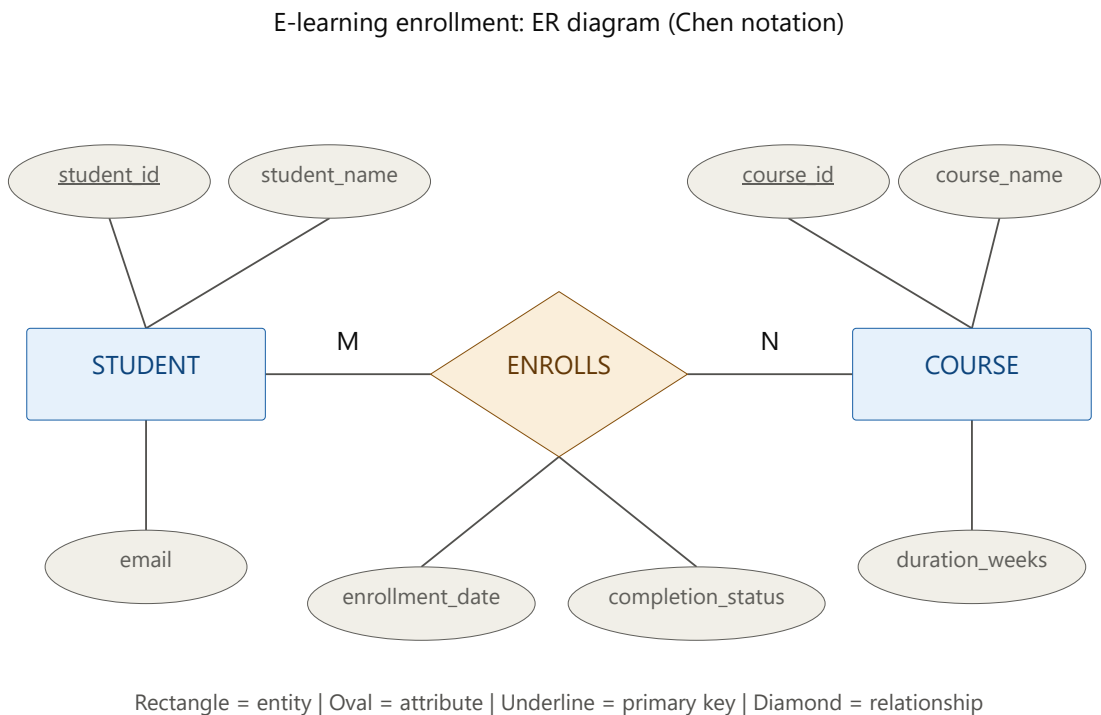


Figure 1: ER diagram of the E-Learning Enrollment database

Relationships

Relationship	Cardinality	Implemented by
students to enrollments	1 : many	enrollments.student_id (FK,

Relationship	Cardinality	Implemented by
		NOT NULL)
courses to enrollments	1 : many	enrollments.course_id (FK, NOT NULL)
students to courses	many : many	enrollments (junction table) with UNIQUE (student_id, course_id)

3. Database Setup

The database elearning_db was created in PostgreSQL and worked on from VS Code using the PostgreSQL extension. schema.sql was run first to create the tables, then db.sql to load the sample data, then query.sql for the queries.

3.1 schema.sql

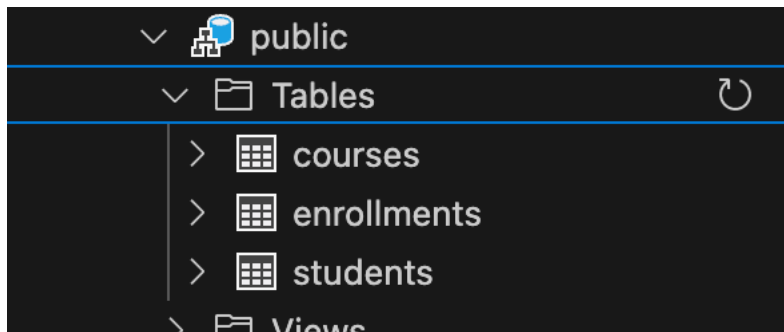
```
-- E-Learning Student Course Enrollment Database

DROP TABLE IF EXISTS enrollments;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS students;

CREATE TABLE students (
    student_id    INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_name  VARCHAR(100) NOT NULL,
    email         VARCHAR(100) UNIQUE
);

CREATE TABLE courses (
    course_id     INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    course_name   VARCHAR(100) NOT NULL UNIQUE,
    duration_weeks INT
);

-- Junction table: resolves the many-to-many relationship
CREATE TABLE enrollments (
    enrollment_id  INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    student_id     INT NOT NULL,
    course_id      INT NOT NULL,
    enrollment_date DATE NOT NULL,
    completion_status VARCHAR(20) NOT NULL DEFAULT 'Not Started',
    CONSTRAINT fk_enroll_student FOREIGN KEY (student_id)
        REFERENCES students(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_enroll_course FOREIGN KEY (course_id)
        REFERENCES courses(course_id) ON DELETE CASCADE,
    CONSTRAINT uq_student_course UNIQUE (student_id, course_id),
    CONSTRAINT chk_status CHECK (completion_status IN ('Not Started', 'In Progress',
'Completed'))
);
```



3.2 db.sql and row counts

db.sql inserts 8 students, 4 courses and 17 enrollment records. Anaya, Kabir and Riya are the three students named in the case study. The other five students and the fourth course (Data Structures) were added so that every query has meaningful output.

```
INSERT INTO students (student_name, email) VALUES
('Anaya', 'anaya@example.com'),
('Kabir', 'kabir@example.com'),
('Riya', 'riya@example.com'),
('Arjun', 'arjun@example.com'),
('Meera', 'meera@example.com'),
('Rohan', 'rohan@example.com'),
('Sneha', 'sneha@example.com'),
('Vikram', 'vikram@example.com');

INSERT INTO courses (course_name, duration_weeks) VALUES
('SQL Fundamentals', 4),
('Python Programming', 8),
('Web Development', 10),
('Data Structures', 12); -- no enrollments (used for Query 4)

INSERT INTO enrollments (student_id, course_id, enrollment_date, completion_status)
VALUES
(1, 1, '2025-01-10', 'Completed'),
(1, 2, '2025-01-15', 'In Progress'),
(1, 3, '2025-02-01', 'In Progress'),
(2, 1, '2025-01-12', 'In Progress'),
(2, 2, '2025-02-05', 'Not Started'),
(3, 1, '2025-01-20', 'Completed'),
(3, 3, '2025-02-10', 'In Progress'),
(4, 1, '2025-01-08', 'Completed'),
(4, 2, '2025-01-18', 'Completed'),
(4, 3, '2025-02-03', 'In Progress'),
(5, 2, '2025-01-25', 'In Progress'),
(6, 1, '2025-01-22', 'In Progress'),
(6, 3, '2025-02-12', 'Not Started'),
(7, 1, '2025-01-14', 'Not Started'),
(7, 2, '2025-01-28', 'Completed'),
(7, 3, '2025-02-06', 'Completed'),
(8, 3, '2025-02-15', 'In Progress');
```

The following query confirms the data volume:

```
SELECT 'students' AS table_name, COUNT(*) AS row_count FROM students UNION ALL
SELECT 'courses', COUNT(*) FROM courses UNION ALL
SELECT 'enrollments', COUNT(*) FROM enrollments;
```

Table	Rows
students	8
courses	4
enrollments	17

4. Design Decisions and Rejected Alternatives

Decision	Chosen	Alternative rejected	Reason
Student-course link	Separate enrollments junction table	Comma-separated list of courses inside students	A list cannot be validated, joined, counted or updated per course. The junction table allows many courses per student and many students per course.
Relationship data	enrollment_date and completion_status stored in enrollments	Storing them in students or courses	These values describe one student in one course, so they belong to the relationship, not to either entity.
Duplicate prevention	UNIQUE (student_id, course_id)	Relying on application code	A student cannot enroll in the same course twice, and the database enforces this for every client.
Junction table key	Surrogate key enrollment_id plus the UNIQUE pair	Composite primary key (student_id, course_id)	A single-column key is simpler to reference and update. The UNIQUE constraint still guarantees one row per pair.
Status values	CHECK constraint (Not Started, In Progress, Completed)	Free text	Free text allows spelling variants that break counting and filtering.

Decision	Chosen	Alternative rejected	Reason
Deleting parents	ON DELETE CASCADE	Blocking deletes (RESTRICT)	Enrollment rows are meaningless without their student or course, so they are removed with it.
Auto numbering	GENERATED ALWAYS AS IDENTITY	MySQL AUTO_INCREMENT or legacy SERIAL	Identity columns are the standard PostgreSQL form, and AUTO_INCREMENT is not valid in PostgreSQL.
Sample data	Extra students and a course with no enrollments	Only the three students and three courses	With only three students, the query for students in more than two courses and the query for courses with no enrollments would return nothing.

5. Queries and Output

Each query is shown with its requirement, the SQL, the reason it was written that way, the result, and a place for the screenshot of the actual output from VS Code.

Query 1. Every student's enrolled courses

Requirement: Display every student's enrolled courses with the enrollment date and completion status.

```
SELECT s.student_name,
       c.course_name,
       e.enrollment_date,
       e.completion_status
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c     ON e.course_id = c.course_id
ORDER BY s.student_name, e.enrollment_date;
```

Why written this way: The student and course names live in separate tables, so the junction table must be joined to both. An INNER JOIN is correct because only existing enrollments are wanted. A student with no enrollments would not appear; use a LEFT JOIN from students if those should be listed too.

Result: 17 rows (Anaya 3, Arjun 3, Kabir 2, Meera 1, Riya 2, Rohan 2, Sneha 3, Vikram 1)

	student_name ↕	course_name ↕	enrollment_date ↕	completion_status ↕
1	Anaya	SQL Fundamentals	2025-01-10	Completed
2	Anaya	Python Programming	2025-01-15	In Progress
3	Anaya	Web Development	2025-02-01	In Progress
4	Arjun	SQL Fundamentals	2025-01-08	Completed
5	Arjun	Python Programming	2025-01-18	Completed
6	Arjun	Web Development	2025-02-03	In Progress
7	Kabir	SQL Fundamentals	2025-01-12	In Progress
8	Kabir	Python Programming	2025-02-05	Not Started
9	Meera	Python Programming	2025-01-25	In Progress
10	Riya	SQL Fundamentals	2025-01-20	Completed
11	Riya	Web Development	2025-02-10	In Progress
12	Rohan	SQL Fundamentals	2025-01-22	In Progress
13	Rohan	Web Development	2025-02-12	Not Started
14	Sneha	SQL Fundamentals	2025-01-14	Not Started
15	Sneha	Python Programming	2025-01-28	Completed
16	Sneha	Web Development	2025-02-06	Completed
17	Vikram	Web Development	2025-02-15	In Progress

Query 2. Enrollments per course

Requirement: Count the enrollments for each course.

```
SELECT c.course_name,
       COUNT(e.enrollment_id) AS total_enrollments
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.course_name
ORDER BY total_enrollments DESC, c.course_name;
```

Why written this way: LEFT JOIN keeps courses that have no enrollments, and COUNT(e.enrollment_id) counts only real enrollment rows, so such a course shows 0. COUNT(*) would wrongly show 1 for an empty course because the NULL-filled joined row still counts as a row. The second sort column keeps equal counts in a stable order.

Result: 4 rows

course_name	total_enrollments
SQL Fundamentals	6
Web Development	6
Python Programming	5
Data Structures	0

Results		Messages
	course_name ↑↓	total_enrollments ↑↓
1	SQL Fundamentals	6
2	Web Development	6
3	Python Programming	5
4	Data Structures	0

Query 3. Students enrolled in more than two courses

Requirement: Find students enrolled in more than two courses.

```
SELECT s.student_name,
       COUNT(e.course_id) AS courses_enrolled
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.student_name
HAVING COUNT(e.course_id) > 2
ORDER BY s.student_name;
```

Why written this way: The condition is on an aggregate, so it must be in HAVING. WHERE runs before grouping and cannot use COUNT. Grouping by student_id as well as student_name keeps two students with the same name separate.

Result: 3 rows

student_name	courses_enrolled
Anaya	3
Arjun	3
Sneha	3

Results		Messages
	student_name	courses_enrolled
1	Anaya	3
2	Arjun	3
3	Sneha	3

Query 4. Courses with no enrollments

Requirement: Find courses that have no enrollments.

```
-- Method 1: LEFT JOIN anti-join
SELECT c.course_id, c.course_name
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
WHERE e.enrollment_id IS NULL;

-- Method 2: NOT EXISTS (alternative)
SELECT c.course_id, c.course_name
FROM courses c
WHERE NOT EXISTS (
    SELECT 1 FROM enrollments e WHERE e.course_id = c.course_id
);
```

Why written this way: LEFT JOIN plus IS NULL is an anti-join: courses with no matching enrollment survive the join with NULLs. NOT EXISTS gives the same result with similar speed. NOT IN is riskier because a NULL in the subquery would silently return nothing, and an INNER JOIN would remove exactly the rows that are wanted.

Result: 1 row (both methods): course_id 4, Data Structures

Results		Messages
	course_id	course_name
1	4	Data Structures

[Insert screenshot: Output of Query 4, Method 2]

Query 5. Update course completion status

Requirement: Update the completion status of a student's course.

```
-- Kabir's Python Programming: Not Started -> In Progress
UPDATE enrollments
SET completion_status = 'In Progress'
WHERE student_id = (SELECT student_id FROM students WHERE student_name = 'Kabir')
  AND course_id = (SELECT course_id FROM courses WHERE course_name = 'Python
Programming');

-- Anaya's Web Development: In Progress -> Completed
UPDATE enrollments
SET completion_status = 'Completed'
WHERE student_id = (SELECT student_id FROM students WHERE student_name = 'Anaya')
  AND course_id = (SELECT course_id FROM courses WHERE course_name = 'Web
Development');

-- Verify the updates
SELECT s.student_name, c.course_name, e.completion_status
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN courses c ON e.course_id = c.course_id
ORDER BY s.student_name, c.course_name;
```

Why written this way: The subqueries look up the ids from names, so the statement stays readable and does not depend on hard-coded ids. The WHERE clause uses both ids so that exactly one enrollment row changes. Without a WHERE clause every enrollment would be overwritten. The CHECK constraint rejects any status other than the three allowed values.

Result: Each UPDATE reports 1 row updated. The verification query returns 17 rows, with Kabir's Python Programming now In Progress and Anaya's Web Development now Completed.

Results

Messages

localhost | elearning_db | schema: session default

	student_name ↑↓🔍	course_name ↑↓🔍	completion_status ↑↓🔍
1	Anaya	Python Programming	In Progress
2	Anaya	SQL Fundamentals	Completed
3	Anaya	Web Development	Completed
4	Arjun	Python Programming	Completed
5	Arjun	SQL Fundamentals	Completed
6	Arjun	Web Development	In Progress
7	Kabir	Python Programming	In Progress
8	Kabir	SQL Fundamentals	In Progress
9	Meera	Python Programming	In Progress
10	Riya	SQL Fundamentals	Completed
11	Riya	Web Development	In Progress
12	Rohan	SQL Fundamentals	In Progress
13	Rohan	Web Development	Not Started
14	Sneha	Python Programming	Completed
15	Sneha	SQL Fundamentals	Not Started
16	Sneha	Web Development	Completed
17	Vikram	Web Development	In Progress

6. Many-to-Many Relationships and Junction Tables

A relational table cannot store a many-to-many relationship directly. If students stored a list of courses, the column would hold several values, which breaks the first normal form. If courses stored a list of students, the same problem would appear on the other side.

The standard solution is a junction table. enrollments holds one row per student-course pair, and each row carries a foreign key to each parent table. This turns one many-to-many relationship into two one-to-many relationships, and the UNIQUE (student_id, course_id) constraint ensures each pair appears only once.

The junction table also gives the relationship a place to keep its own attributes. The enrollment date and the completion status are facts about a student in a particular course, so they are stored in enrollments. The schema has no partial or transitive dependencies, so it satisfies third normal form.

7. Limitations and Future Improvements

- Only the latest completion status is stored. A history table would record when each status change happened.
- Completion is a three-value status; a percentage-progress column would give finer tracking.
- There is no instructor, category, price or certificate information for courses.
- Re-enrolling after dropping a course is not supported because of the UNIQUE pair; a dropped flag or a history table would allow it.

- Status values are stored as text with a CHECK constraint; a lookup table would make adding statuses easier.
- Indexes on enrollments.student_id and enrollments.course_id would help if the platform grows to thousands of learners.

Conclusion

The database gives a simple, normalized model of an e-learning platform. Primary keys, foreign keys, the UNIQUE pair and CHECK constraints keep enrollment records consistent, and the enrollments junction table implements the many-to-many relationship between students and courses while storing the enrollment date and completion status. The queries cover joins, aggregation with HAVING, anti-joins, NOT EXISTS and updates, and each design choice is recorded together with the alternative that was rejected.